

B.Tech. CSE (III YEAR - VI SEM) (2025-2026)

DEPARTMENT OF COMPUTER ENGINEERING & APPLICATIONS



GLA University

**17km Stone, NH-19, Mathura-Delhi Road P.O. Chaumuhan, Mathura -
281406 (Uttar Pradesh) India**

Project Title:

AI-Based Voice-Enabled Intelligent System Assistant

Team Member 1:

Vansh Raghav

Uni. Roll No.: 2315510233

Team Member 2:

Uday Bansal

Uni. Roll No.: 2315510220

Team Member 3:

Swayam Adhana

Uni. Roll No.: 2315510214

Mentor:

Mr Preshit Mangesh Desai

INDEX	
Page No.	Content
1	NA
2	Index
3	Cover
4	Overview
5 – 7	Scope & Control
8	Market & Competitors
9	Team & Roles
10 – 11	Workplan & Assignments
12	Users & UX
13	Business and Technical Objective
14	Technology & Architecture
15	UI & UX, Acceptance Criteria
16	Quality and Testing
17	Security and Compliances
18	Risk and Mitigations
19	R & D Plan
20	References

Project Synopsis: AI-Based Voice-Enabled Intelligent System Assistant

0. Cover

- **Project title:** AI-Based Voice-Enabled Intelligent System Assistant (OmniAssist AI)
- **Team ID:** T19
- **Institute / Course:** GLA University / B.Tech Computer Science (AI/ML & IoT)
- **Version:** v1.0
- **Date:** 3 Feb 2026

Revision history

Version	Date	Author	Change
v0.1	Feb 1, 2026	Vansh Raghav	Initial draft: project outline and planning.
v1.0	Feb 3, 2026	Vansh Raghav	Final revision: diagrams, and references included.

1. Overview

Problem statement: With the rapid growth of computing power and software complexity, human–computer interaction has not evolved proportionally. Users still rely heavily on manual navigation, keyboard-driven workflows, and fragmented applications to perform routine tasks. While modern AI assistants exist, they are often limited to predefined commands, lack deep system-level awareness, and fail to understand user intent in complex or multi-step scenarios.

Most existing assistants operate as isolated tools rather than integrated system companions. They do not maintain long-term context, cannot reliably execute system-level tasks, and provide limited personalization.

This gap prevents AI from functioning as a true intelligent assistant capable of proactively supporting users in everyday computing activities.

There is a need for an intelligent, voice-enabled system assistant that can understand natural language, reason

over user intent, interact directly with the operating system, and assist users in a seamless, human-like manner

while respecting privacy and permission boundaries.

Goal: The goal of this project is to design and develop **OmniAssist AI**, an AI-based voice-enabled intelligent system assistant that acts as a proactive digital companion for personal computers. The system aims to combine natural language understanding, speech interaction, reasoning capabilities, and controlled system-level access to help users perform tasks efficiently and intuitively.

The project will implement a modular, agent-based architecture powered by Large Language Models (LLMs), enabling the assistant to interpret user intent, plan multi-step actions, and execute tasks such as file

management, application control, information retrieval, and productivity assistance through natural voice conversations. The final outcome will be a working major-project prototype capable of operating in both

offline (local AI) and online (cloud-assisted) modes.

Non-goals:

- Full autonomous control without user permissions is out of scope; all actions require explicit approval or predefined access rules.
- No enterprise-scale system administration (server orchestration, remote device control).
- No direct integration with proprietary or restricted operating-system internals.

- No biometric authentication or medical/legal decision-making.
- No replacement of human judgment in critical tasks.

Value proposition: OmniAssist AI differentiates itself by functioning as a **system-aware intelligent assistant** rather than a simple chatbot or voice command tool. Unlike conventional assistants that rely on fixed commands or cloud-only processing, OmniAssist AI combines local intelligence, contextual memory, and real-time reasoning to provide reliable, privacy-aware assistance.

The system improves productivity by reducing repetitive actions, enhances accessibility through hands-free voice interaction, and creates a more natural human-computer relationship. By integrating conversational AI with direct system interaction, OmniAssist AI demonstrates how generative AI can be transformed into a practical, trustworthy, and user-centric computing companion.

2. Scope and Control

2.1 In-scope

1. Voice-Based Interaction:

Natural, hands-free communication using speech-to-text and text-to-speech technologies, allowing users to interact with the system through conversational voice commands.

2. Natural Language Understanding and Reasoning:

Integration of Large Language Models (LLMs) to interpret user intent, understand multi-step requests, and generate appropriate responses or action plans.

3. Agent-Based Task Execution:

An intelligent agent layer capable of breaking down complex user requests into smaller actionable steps

and executing them sequentially.

4. System-Level Assistance (Permission-Based):

Controlled access to operating system functionalities such as opening applications, managing files and

folders, searching local data, and handling basic system workflows with user approval.

5. Contextual Memory:

Short-term and session-level memory to maintain conversational context, allowing follow-up instructions

and partial commands to be understood correctly.

6. Hybrid Operation Mode:

Support for both **offline (local AI models)** and **online (cloud-assisted AI services)** operation, ensuring functionality even without continuous internet connectivity.

7. Real-Time Information Retrieval:

Ability to fetch up-to-date information (e.g., weather, reminders, general knowledge) when internet access

is available.

8. User Interface Layer:

A simple desktop or web-based interface displaying assistant responses, task confirmations, logs, and basic settings.

2.2 Out-of-scope

1. Fully autonomous decision-making without user confirmation.
2. Administrative-level system control (e.g., kernel-level operations).
3. Multi-user enterprise management or shared system environments.
4. Native mobile application development.
5. Integration with third-party proprietary platforms requiring paid licenses.
6. Advanced emotional or psychological diagnosis features.

2.3 Assumptions

1. Users provide explicit permission for system-level operations.
2. The system is deployed on a personal computer with microphone and speaker

support.

3. Local hardware is capable of running lightweight AI models.
4. Internet connectivity is available for cloud-based features when required.
5. The assistant operates on a single-user system environment.
6. Project duration is limited to one academic semester.
7. Limited availability of high-end GPU resources for training or inference.
8. Computational constraints for running large models locally.
9. Development is carried out by students with academic-level resources.
10. Privacy and security requirements restrict unrestricted system access.

2.4 Dependencies

- Software Libraries: Python, Flask, Speech Recognition, LLM frameworks and OS-Level automation tools.
- AI Model: Local Language models for offline reasoning and cloud-based LLM APIs for advanced processing.
- Hardware: Microphone, speakers and standard personal computing hardware.
- Services: Cloud inference APIs and real time information sources.

2.6 Acceptance criteria and sign-off

- The assistance accurately understand and responds to natural voice commands.
- Multi step tasks are executed correctly under user authorization.
- The system maintains conversational context across interaction.
- Core functionality operate in both offline and online modes.

Sign-off table

Stakeholder	Role	Decision area	Signature/Approval	Date
Mr. Preshit Desai	Mentor	Requirements, scope, final approval	Approved	3 Feb 2026
Vansh Raghav	Team Lead	Design, integration, deliverables	Approved	3 Feb 2026

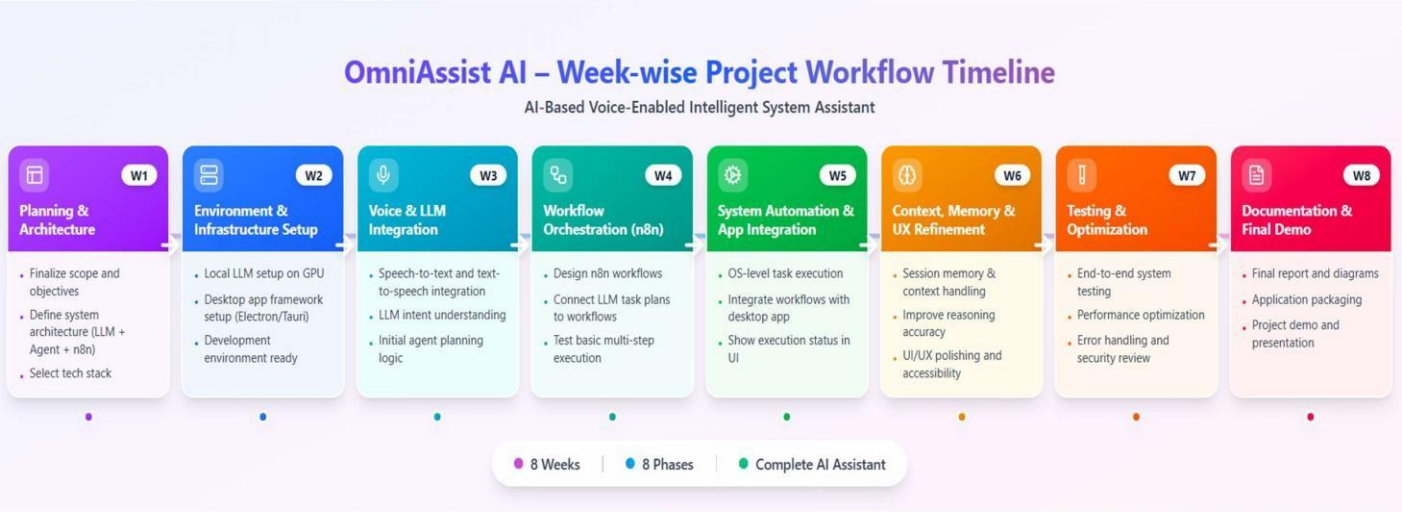
3. Team and Roles

Member	Role	Responsibilities	Key skills
Vansh raghav	Team Lead /System design	Scope, backlog, reviews	Product, APIs
Uday Bansal	Tech Lead C Backend	Arch, APIs, security	Node, Express, SQL, FastAPI
Swayam Adhana	Frontend C AI Engineer	React UI, state, a l l y, AI performance Optimization	React, TS, Pytorch

4. Week-wise Plan and Assignments

Week	Milestones	Vansh (Lead C Arch)	Swayam (AI C LLM)	Uday (UI/ App)	Deliverables	Status
1	Project Initiation freeze	Finalize scope, system architecture, C tech stackscope	Research local LLMs C GPU requirements	Research desktop frameworks (Electron/Tauri), Wireframes	Architecture Doc, Tech Stack Finalized	Planned
2	Core Infrastructure	Design LLM → n8n → OS flow; define permissions	Setup local LLM on RTX 5050; baseline inference	Setup desktop app project; base UI layout	Dev environment ready, Baseline demos	Planned
3	Voice C LLM Integration	Define intent schema C agent planning logic	Integrate Speech-to-Text with LLM reasoning	Integrate voice UI (mic, status, responses)	Voice → Text → LLM working	Planned
4	n8n Workflow Design	Design core n8n workflows (File mgmt, App launch)	Generate structured task plans for n8n	UI for task confirmation C workflow status	n8n workflows + API integration	Planned
5	Automation Integration	Integrate n8n with OS-level scripts/APIs	Optimize LLM outputs for multi-step tasks	Display execution logs C progress in app	End-to-end task execution demo	Planned

6	Context C Refinement	Design context memory C session handling	Improve reasoning with memory/follow-ups	UX refinement C accessibility improvements	Context-aware assistant prototype	Planned
7	Testing C Optimization	System testing, security review, failure handling	Performance optimization C GPU tuning	UI polish C error handling	Stable beta version	Planned
8	Documentation C Demo	Final report C architecture diagrams	AI module documentation C results	App packaging C deployment build	Final submission C Demo	Planned



5. Users and UX

5.1 Personas

- **Primary User:** Seeks speed, convenience, and hands-free interaction for daily tasks (files, apps, productivity).
- **Secondary:** Values transparency, workflow inspection, and system control.

6.2 Top user journeys

- Voice Input → Intent Confirmation → n8n Execution → UI/Voice Feedback
- Compound Instruction → LLM Planning → n8n Orchestration → Real-time Progress UI
- General Query → Contextual Retrieval → Speech/Text Response → Follow-up Handling

6.3 User stories

- "I want to interact with my computer via **natural voice** for hands-free tasks."
- "I want the assistant to remember **context** for follow-up instructions."
- "I want to see **real-time logs** and provide **confirmation** for critical actions."

6.4 Accessibility s localization

- The assistant accurately understands and responds to natural voice commands.
- Multi-step tasks are executed correctly under user authorization.
- The system maintains conversational context across interactions.
- Core functionalities operate in both offline and online modes.
- No critical system failures or unhandled errors occur during operation.

7. Market and Competitors

7.1 Competitor table

Solution/Assitant	Approach	Platform	Capabilities	Limitations
-------------------	----------	----------	--------------	-------------

Google Assistant	Cloud-based AI assistant	Mobile, Web	Voice interaction, web queries, smart-device control	Limited system-level PC control, privacy concerns
Microsoft Copilot	LLM-powered assistant	Windows OS	Application integration, document assistance	Restricted to Microsoft ecosystem
Open-source Voice Assistants	Rule-based / hybrid	Desktop/Linux	Customizable, offline options	Limited reasoning and natural conversation

7.2 Positioning

OmniAssist AI is positioned as a **next-generation intelligent system assistant** focused on productivity, accessibility, and human-like interaction. Unlike traditional assistants that respond to isolated commands, OmniAssist AI emphasizes:

- Intent-driven task execution
- Context-aware conversation
- Hybrid offline–online functionality
- User-controlled system access

This positioning makes it suitable for students, developers, and professionals seeking a practical AI companion for daily computing tasks.

8. Objectives and Success Metrics

- **O1: Accurate Intent Understanding.** Achieve reliable interpretation of

natural language voice commands, correctly identifying user intent in at least 90% of evaluated test commands. **KPI:** Intent recognition accuracy measured on a predefined command set; correct task mapping rate.

- **O2: Multi-Step Workflow Execution via n8n.** Ensure successful execution of complex, multi-step user tasks using agent-based planning and n8n workflow orchestration for at least 85% of evaluated workflows. **KPI:** Workflow completion rate; number of successfully executed end-to-end tasks without manual correction.
 - **O3: Responsive End-to-End Interaction.** Ensure the end-to-end response time (voice command to action confirmation) is ≤ 3 seconds for the 90th percentile under normal system load. **KPI:** p95 latency measured from voice input to UI/voice response.
 - **O4: Deployable Desktop Application.** Deliver a fully functional **standalone desktop application** that launches like a normal laptop app and integrates voice interaction, AI reasoning, and workflow execution without critical failures. **KPI:** Successful completion of predefined task scenarios (pass/fail acceptance test); crash-free execution during demo.
 - **O5: Hybrid Offline and Online Operation.** Enable the assistant to perform core tasks using local LLM inference while seamlessly switching to online services when internet connectivity is available. **KPI:** Successful task execution rate in offline mode; fallback success rate when switching to online mode.
-

9. Architecture

- **High-Level Architecture:**

- **User Interaction (Desktop Application):**

A standalone cross-platform desktop application (Electron/Tauri) that launches like a normal laptop app. Captures user voice input, displays assistant responses, workflow confirmations, and execution status.

- **Speech Processing Module:**
Includes Speech-to-Text (STT) for converting user voice commands into text and Text-to-Speech (TTS) for generating spoken responses, enabling hands-free interaction.
- **AI Reasoning Engine (LLM):**
A locally hosted Large Language Model executed with GPU acceleration (RTX 5050) for intent understanding, contextual reasoning, and response generation. Generates structured task plans rather than executing system commands directly.
- **Agent-Based Task Planner:**
Interprets LLM outputs and decomposes high-level user intents into structured, multi-step task plans with defined parameters, dependencies, and safety checks.
- **Workflow Orchestration Engine (n8n):**
Executes multi-step task plans using predefined workflows. Handles conditional logic, retries, error handling, logging, and ensures controlled execution of system actions.
- **System Automation Layer:**
Performs permission-based system operations such as file management, application launching, scheduling, and API interactions using OS-level scripts and automation tools.
- **Backend Services (FastAPI):**
A Python FastAPI server that coordinates communication between the desktop application, LLM engine, agent planner, and n8n workflows. Handles business logic, permissions, and execution flow.
- **Context and Memory Module:**
Maintains session-level conversational context and task state to support follow-up commands and continuous interaction.
- **Security and Permission Control:**
Implements user confirmations, access restrictions, and audit logging to ensure safe and transparent system interaction.

- **Deployment:**
The application is packaged and deployed as a local desktop application. Backend services and n8n workflows run locally, with optional cloud integration for enhanced AI reasoning and real-time information retrieval.

10. Technical Workflow Diagram



11. Quality: NFRs and Testing

11.1 Non-functional requirements

Metric	Target (SLO)	Measurement
Intent Accuracy	≥ 90%	Test command set evaluation
Execution Success	≥ 85%	n8n end-to-end logs

Error rate	$p95 \leq 5s$	Voice-to-Action confirmation timing
System Uptime	$\geq 99\%$	Runtime monitoring during testing
Safety	0 Critical Violations	Manual review of permission flows

11.2 Test plan

Area	Type	Tools	Owner
AI Reasoning	Functional	PyTest, Custom Prompts	Swayam
Workflows	Integration	n8n Logs	Vansh
Backend API	Unit	PyTest, Postman	Vansh
Desktop App	UI / E2E	Manual / App Logs	Uday
Automation	System	OS Scripts	Joint Team

12. Security and Compliance

12.1 Threat model (STRIDE)

- Malicious Inputs:

Users may provide malformed voice input, unintended commands, or attempts to trigger unsafe system actions. Mitigation includes input validation, intent verification, and confirmation prompts before executing sensitive tasks.

- Unauthorized System Actions:

AI-generated actions could unintentionally perform unsafe operations. Mitigation includes a permission-based execution model, user confirmations, and workflow constraints enforced through **n8n**.

- **Denial of Service:**

Repeated or complex voice commands could overload system resources. Mitigation includes request throttling, workflow execution limits, and local resource monitoring.

- **Data Privacy and Breach:**

The system does not process sensitive personal data. Voice input and context are processed locally by default. No user credentials are stored. Any data transmitted to online services uses secure communication channels.

12.2 AuthN/AuthZ

- **Authentication (AuthN):**

No mandatory user authentication is required for the MVP desktop application. If extended in future versions, a simple local user authentication mechanism (e.g., OS-based identity or email/password with JWT) may be implemented.

- **Authorization (AuthZ):**

Role-based access control is not required for the MVP. Authorization is enforced through **explicit user confirmations** and permission boundaries for system-level actions.

12.3 Audit and logging

- **Logging:**

The system logs key events including voice commands (text form), generated task plans, workflow execution status, system actions, and errors with timestamps for debugging and evaluation.

- **Retention:**

Logs and any stored context information will be retained only for the duration of the project (≤ 1 year) and then securely deleted. No long-term user data retention is required.

- **Monitoring:**

Basic monitoring of system uptime, workflow failures, and error rates will be performed during development and testing phases.

13. Delivery and Operations

13.1 Release plan

The project will be released in clearly defined phases to ensure stability and progressive integration.

- Alpha Release: Core components operational, including voice input, basic LLM reasoning, and simple task execution.
 - Beta Release: Full integration of the desktop application, agent-based planning, and n8n workflow orchestration with stable end-to-end task execution.
 - Final Release (MVP): Complete OmniAssist AI system with refined UI/UX, hybrid offline–online support, security controls, documentation, and demonstration-ready build.
- **Cloud Demo:** The system will be deployed primarily as a **standalone desktop application** that can be installed and launched on a personal laptop. Backend services and n8n workflows will run locally on the same system. Optional cloud-based AI services may be enabled for demonstration of online capabilities.
 - **Presentations:** Basic operational procedures will be defined for starting, stopping, and monitoring the application, backend services, and workflow engine. Logs will be used to identify errors and performance issues during testing and demonstration.
-

14. Risks and Mitigations

Risk	Impact	Score	Mitigation	Owner
Intent Misinterpretation	High	6	Confirmation prompts for sensitive tasks	Vansh
n8n Workflow Failure	High	6	Robust error handling C retry logic	Vansh

Integration Issues	Medium	4	Early API contract definition C syncs	Uday
Schedule Delays	High	6	Strict week-wise planning C scope freeze	Vansh
GPU Constraints	High	6	Quantized models C CPU fallback	Swayam

15. Research and Evaluation

- **Performance Evaluation:** We will evaluate the performance of OmniAssist AI by measuring intent understanding accuracy, workflow execution success rate, and end-to-end response latency. These metrics will be compared against baseline AI assistants and open-source voice assistant frameworks to assess improvements in contextual reasoning and task automation.
- **Reasoning and Planning Evaluation:** The quality of LLM-based reasoning and agent planning will be evaluated by analyzing how accurately complex user commands are decomposed into structured, multi-step workflows. Success will be measured by mentor review of task plans and correctness of executed actions.
- **Usability Testing:** Informal user testing will be conducted with peers and lab members. Metrics such as time from voice command to task completion, ease of use, clarity of confirmations, and overall user satisfaction will be recorded.
- **Limitations:** The current MVP focuses on a single-user desktop environment and predefined system tasks. It does not support multi-user profiles, advanced personalization₀, or enterprise- scale deployment. Performance may vary depending on local hardware capabilities.
- **Future Work:** Future enhancements may include long-term personalized memory, proactive task suggestions, multi-device synchronization, advanced natural language understanding, and deployment optimizations for scalability. Integration with additional system services and more sophisticated workflow intelligence may also be explored.

16. Appendices

- **Glossary:**

- **LLM (Large Language Model):**

A deep learning–based language model capable of understanding, generating, and reasoning over natural language text. Used in OmniAssist AI for intent understanding and task planning.

- **Agent-Based System:**

A software architecture where an intelligent agent interprets goals, plans actions, and executes tasks autonomously or semi-autonomously under defined constraints.

- **Speech-to-Text (STT):**

A technology that converts spoken language into text, enabling voice-based interaction with the system.

- **Text-to-Speech (TTS):**

A technology that converts textual responses into spoken audio, allowing the assistant to respond verbally.

- **n8n Workflow Engine:**

An open-source workflow orchestration tool used to execute multi-step tasks with conditional logic, retries, and logging.

- **Desktop Application:**

A standalone application installed and launched on a personal computer, providing tighter system integration than browser-based applications.

- **API (Application Programming Interface):**

A set of endpoints that enable communication between system components such as the desktop application, backend services, and workflow engine.

- **UI (User Interface):**

The graphical interface through which users interact with OmniAssist AI, including voice controls, confirmations, and task status displays.

- **MVP (Minimum Viable Product):**

The simplest functional version of OmniAssist AI that fulfills core requirements such as voice interaction, task execution, and workflow orchestration.

- **KPI (Key Performance Indicator):**

A measurable value used to evaluate system performance, such as response latency, workflow success rate, or user satisfaction.

- **References:**

- Vaswani et al., 2017, "*Attention Is All You Need*," NeurIPS. (Foundation of transformer-based language models used in modern LLMs.)
- Brown et al., 2020, "*Language Models are Few-Shot Learners*," NeurIPS. (Introduced large-scale language models and in-context learning.)
- OpenAI, *Whisper: Robust Speech Recognition via Large-Scale Weak Supervision*. (Reference for speech-to-text implementation.)
- n8n Documentation, *Workflow Automation Platform*. (Reference for workflow orchestration and execution logic.)
- FastAPI Documentation, *High-performance Python web framework for APIs*. Electron / Tauri Documentation, *Cross-platform desktop application frameworks*